# Teste de aspectos e teste apoiado por aspectos

Seminário sobre Testes de Software · Engenharia de Software II (INF 16160) — 2026/2

Capítulo 7 de Introdução ao Teste de Software — Lemos, Ré, Masiero, Ferrari e Cafeo

Bernardo Vargens Broedel · Eduardo Capobiango Ferraz · Mateus Biancardi da Silva · Miguel Zon Murad · Victor Augusto Fernandes Rangel

Prof. Vítor E. Silva Souza · 10 de dezembro de 2026

# Duas perguntas guiam o capítulo

## 1 Como a POA pode ajudar a testar programas orientados a objetos?

Aspectos de desenvolvimento que implementam infraestrutura de teste — imitadores, verificações, instrumentação — sem tocar no código que está sendo testado.

## 2 Como testar programas que usam POA?

Aspectos são uma entidade nova: as técnicas de teste estrutural, baseada em estados e de mutação precisam ser adaptadas, e novos critérios precisam ser definidos.

*As duas perguntas se apoiam na mesma propriedade: o aspecto é um módulo separado que afeta vários pontos do sistema. Isso é uma vantagem quando o aspecto é o código de teste — e um desafio quando é o código sob teste.*

# Roteiro

1. **Fundamentos de POA** — interesses transversais, aspectos, AspectJ — Eduardo
2. **Teste apoiado por aspectos** — cinco usos práticos em programas OO — Bernardo
3. **Testar programas OA: o problema** — o que muda e quais defeitos surgem — Mateus
4. **Teste estrutural de programas OA** — AODU, CoDU, critérios e JaBUTi/AJ — Miguel
5. **Estados, mutação e ordenação** — ASM, operadores, Proteum/AJ e ORD — Victor

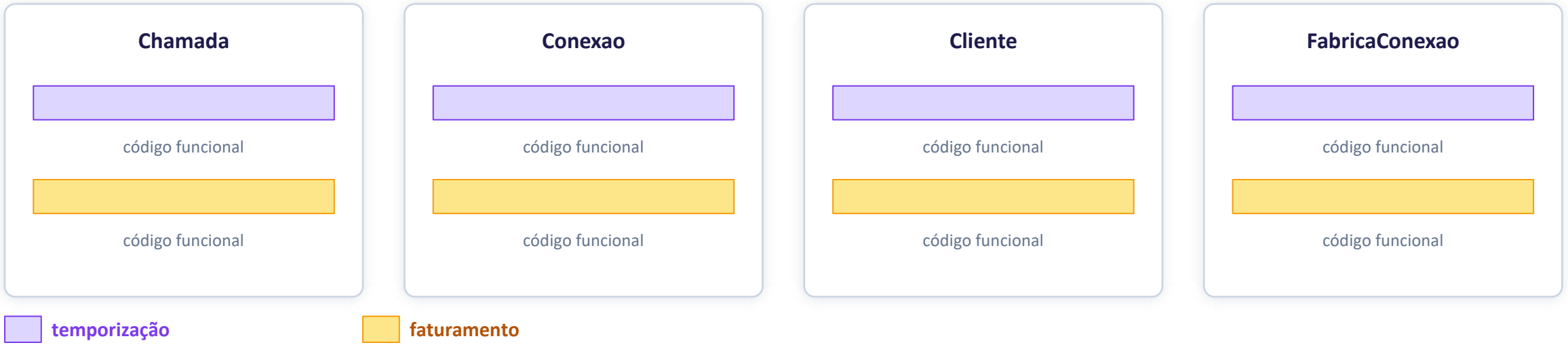
1

# Fundamentos de POA

Interesses transversais, aspectos e a linguagem AspectJ

# O problema: interesses transversais

A orientação a objetos não separa bem certos requisitos — em geral não funcionais — cuja implementação se espalha por vários módulos.




- Espalhamento (scattering): o mesmo interesse aparece em muitos módulos.
- Entrelaçamento (tangling): dentro de um módulo, código funcional e não funcional se misturam.
- Consequência: manutenção cara e difícil de localizar — mexer no faturamento é mexer em todas as classes.

# A proposta da POA

**CÓDIGO-BASE**

Os módulos que implementam as funcionalidades básicas — as classes do sistema. Permanecem sintaticamente inconscientes dos aspectos.

**ASPECTOS**

Módulos isolados que implementam os interesses transversais e têm a capacidade de afetar vários outros módulos do sistema.

**combinação (weaving)**

- Quantificação: um único aspecto contribui para vários pontos do sistema, sem que esses pontos o chamem.
- Inconsciência (obliviousness): os métodos afetados não sabem que existem aspectos — menor acoplamento.
- A combinação pode ser estática (em tempo de compilação) ou dinâmica (em tempo de execução).
- POA não substitui um paradigma: é uma técnica usada junto com os existentes.

# Vocabulário essencial

**Ponto de junção**

*join point*

Ponto identificável na execução do programa: chamada ou execução de método, leitura ou escrita de atributo, tratamento de exceção.

**Conjunto de junção**

*pointcut*

Expressão que seleciona um conjunto de pontos de junção. Composta por designadores (call, execution, get, set, cflow, this, target, args) com &&, || e !.

**Adendo**

*advice*

Bloco de código, parecido com um método, executado antes (before), depois (after) ou no lugar (around) dos pontos de junção selecionados.

**Declaração intertipos**

*inter-type declaration*

Introduz atributos, métodos ou heranças em classes já existentes, a partir do aspecto.

# AspectJ: o aspecto de temporização

Extensão de Java para POA. Exemplo do capítulo: sistema de telefonia, em que temporização, faturamento e transferência de chamadas são aspectos.

```
public aspect Temporizacao {

  private Temporizador Conexao.temporizador = new Temporizador();

  pointcut finalizacao(Conexao c): target(c) &&
      call(void Conexao.desconecta());

  after(Conexao c) returning(): finalizacao(c) {
    getTemporizador(c).parar();
    c.getChamador().tempoTotalConexao += ...;
  }
}
```

**declaração intertipos**

introduz um atributo na classe Conexao

**conjunto de junção**

toda chamada a Conexao.desconecta()

**adendo posterior**

executa quando o ponto de junção retorna

*Os três tipos de adendo: before, after (returning / throwing / sempre) e around — este executa no lugar do ponto de junção e pode retomá-lo com proceed().*

# Dois propósitos para os aspectos

D

## Aspectos de desenvolvimento

- Apoiam atividades como depuração, verificação de padrões e teste.
- Não entram na aplicação final: são retirados antes da entrega.
- São o tema da Parte 2 deste seminário.

P

## Aspectos de produção

- Implementam interesses da própria aplicação (temporização, faturamento, persistência).
- Estão no produto entregue — e, portanto, precisam ser testados.
- São o tema das Partes 3, 4 e 5.

**É essa distinção que divide o capítulo em duas metades: usar aspectos para testar e testar o que usa aspectos.**

# 2

# Teste apoiado por aspectos

Cinco usos da POA para testar programas orientados a objetos

# Por que aspectos ajudam a testar

- O código de apoio ao teste — rastreamento, registro, simulação, verificações — é um interesse transversal como qualquer outro.
- Implementado como aspecto, ele fica separado do código funcional e é inserido e removido sem alterar o código-base.
- Terminado o teste, o aspecto é retirado e o programa original continua intacto: nada de código de teste esquecido em produção.

1

**Imitadores virtuais**

simular dependências sem alterar classes

2

**Teste embutido**

casos de teste dentro de componentes

3

**Padrões de código**

declare error e declare warning

4

**Contratos**

invariantes, pré e pós-condições

5

**Instrumentação**

rastreamento, registro e cobertura

# 1. Imitadores virtuais de objetos

Para testar validar() isoladamente, é preciso simular autenticar(), de que ele depende.

### IMITADOR TRADICIONAL (MOCK)

- Cria-se uma classe imitadora que reescreve o método.
- A classe em teste precisa ser alterada para chamar a imitadora.
- Para variar o retorno por caso de teste, o imitador fica complexo.

### IMITADOR VIRTUAL (ASPECTO)

- Um adendo around intercepta as execuções e devolve o valor simulado.
- Nada muda nas classes do sistema — a característica não invasiva do aspecto.
- Cada caso de teste define o seu próprio retorno, em uma tabela.

```
aspect InterceptadorMetodo {
  pointcut todasAsChamadas(): execution(* *.*(..)) &&
      !within(TesteBaseadoAspectos);

  Object around(): todasAsChamadas() {
    Object v = getValorRetornoImitador(classe, metodo);
    return (v != null) ? v : proceed();
  }
}
```

```
public void testarValidarUsuarioValido() {
  DialogoAcesso dialogo = new DialogoAcesso();

  setImitador("ControladorAcesso", "autenticar",
      new Integer(USUARIO_VALIDO));

  dialogo.validar();
  assertEquals("Usuário autenticado", ...);
}
```

# 2. Teste embutido em componentes

- Testabilidade é um interesse não funcional: pode ser implementada como aspecto.
- O aspecto introduz, por declarações intertipos, os métodos de teste e os atributos de estado no próprio componente.
- O componente não é alterado: o projeto e a implementação ficam livres de código de teste.
- Transições de estado são capturadas por conjuntos de junção e tratadas nos adendos — o que torna observável um estado difícil de alcançar só com POO.

```
public aspect AspectoLista {
  private String ListaUsuarios.estado = "Vazia";

  public void ListaUsuarios.testarInserirUsuario(){
    inserir("João", "senhaJoão");
    if (!apenasUm())
      System.out.println("Problemas!");
  }

  pointcut crescendo(ListaUsuarios lu):
    target(lu) && call(void *.inserir(..));

  after(ListaUsuarios lu): crescendo(lu) {
    // atualiza lu.estado
  }
}
```

Útil para quem desenvolve o componente (testa cada um isoladamente) e para quem integra (testa o componente no ambiente de uso, junto das outras partes do sistema).

# 3. Verificação de padrões de código

Aspectos também verificam regras de projeto e de codificação — em tempo de compilação, sem executar o programa.

**REGRA DE PROJETO — declare error**

```
public aspect AspectoRegras {
  declare error:
    !withincode(* ControladorAcesso.*(..)) &&
    call(public int ListaUsuarios.verificarSenha(..)):
    "Use o método ControladorAcesso.autenticar().";
}
```

Só o ControladorAcesso (ou suas subclasses) pode verificar a senha; qualquer outra chamada vira erro de compilação.

**PADRÃO DE CODIFICAÇÃO — declare warning**

```
public aspect AspectoPadroes {
  declare warning:
    set(!public * *) &&
    !withincode(* set*(..)):
    "Escrevendo atributo fora do método set.";
}
```

Escrever em um atributo não público fora de um método set gera aviso: o encapsulamento é verificado automaticamente.

*Não é teste no sentido estrito: é análise estática barata, escrita na mesma linguagem do sistema e executada pelo próprio compilador AspectJ.*

# 4. Invariantes, pré e pós-condições

Projeto por contrato sem entrelaçar as verificações no código funcional: as condições ficam todas em um aspecto.

```
public aspect AspectoChecagemPrePos {
  pointcut autenticar(String id, String senha):
    call(int ControladorAcesso.autenticar(String, String))
    && args(id, senha);

  before(String id, String senha): autenticar(id, senha) {
    if (id == null || senha == null)          // precondição
      throw new NullPointerException("Parâmetro inválido.");
  }

  after(String id, String senha) returning(int r):
      autenticar(id, senha) {
    if (r != USUARIO_VALIDO && r != NENHUMA_SENHA && ...)
      throw new RuntimeException("Resultado inválido."); // pós
  }
}
```

**Precondição**

o que deve valer para o método poder ser chamado

*adendo before*

**Pós-condição**

o que o método garante ao retornar

*adendo after returning*

**Invariante**

o que vale durante toda a vida do objeto

*adendo sobre set(...)*

# 5. Instrumentação de código

- Coletar informação em tempo de execução é uma aplicação típica da POA: rastreamento, registro e análise de cobertura.
- Rastrear chamadas a um método, com seus parâmetros, ajuda a entender o fluxo e a testar segurança.
- Monitorar a escrita de um atributo responde onde e por que o valor mudou — útil na depuração.
- Terminada a investigação, basta não combinar o aspecto: o código-base nunca foi tocado.

```
public aspect AspectoRastrear {
  pointcut monitorarAtributo(String s, DialogoAcesso da):
    set(private String DialogoAcesso.idUsuario)
    && args(s) && target(da);

  before(String s, DialogoAcesso da):
      monitorarAtributo(s, da) {
    System.out.print("idUsuario alterado de: ");
    System.out.print(da.getIdUsuario());
    System.out.println(" para " + s);
  }
}
```

**RESUMO DA PARTE 2**

Em todos os cinco usos o ganho é o mesmo: o código de teste é escrito uma vez, alcança muitos pontos do sistema por quantificação e sai sem deixar rastro. É o aspecto de desenvolvimento em ação.

3

# Testar programas OA

O que muda na atividade de teste e quais defeitos aparecem

# O que torna o teste de programas OA diferente

POA produz módulos mais coesos e menos acoplados — mas não elimina o erro humano, e um mecanismo mais expressivo também pode ser mal usado. Alexander et al. apontam quatro características:

### 1 Sem existência independente

O aspecto não tem identidade própria: depende do contexto das classes em que é combinado. Não há como executá-lo sozinho.

### 2 Forte acoplamento ao contexto

A implementação do aspecto depende da representação e da implementação interna das classes que ele entrecorta.

### 3 Dependências pouco visíveis

Dependências de controle e de dados não são facilmente compreensíveis a partir do código-fonte dos aspectos nem das classes.

### 4 Comportamento emergente

O defeito pode estar na classe, no aspecto — ou ser efeito colateral de uma ordem particular de combinação de vários aspectos.

# Taxonomia de defeitos de Alexander et al.

Seis classes, baseadas no mau uso dos mecanismos da POA. Os critérios e as estratégias de teste devem atacar uma ou mais delas.

**1 Restrição incorreta em padrões de conjuntos de junção**

o conjunto de junção é restritivo demais ou de menos: captura menos (ou mais) pontos do que deveria

**2 Precedência incorreta de aspectos**

dois aspectos afetam pontos de junção coincidentes e a ordem entre eles é definida de forma errada

**3 Falha na preservação de pós-condições**

um adendo altera dados do componente afetado e viola a pós-condição do método

**4 Falha na preservação de invariantes de estado**

o aspecto quebra uma invariante predefinida do objeto que entrecorta

**5 Foco incorreto no fluxo de controle**

erros em conjuntos de junção cflow: pontos negligenciados ou tomados por engano

**6 Mudanças incorretas em dependências de controle**

o mau uso de aspectos altera erroneamente as dependências de fluxo de controle

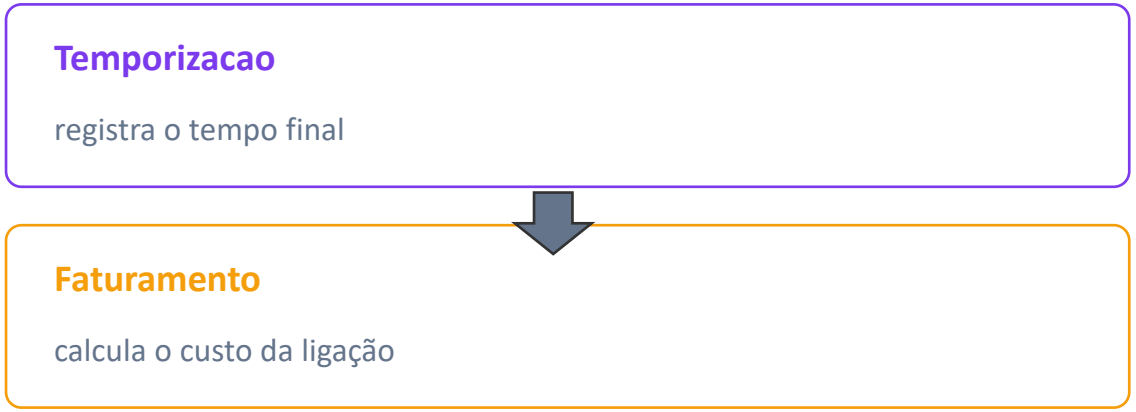
# Dois defeitos no exemplo da telefonia

### CLASSE 1 — CONJUNTO DE JUNÇÃO RESTRITIVO DEMAIS

```
pointcut finalizacao(Conexao c): target(c) &&
    call(void Conexao.desconecta());
```

- Captura as chamadas a desconecta().
- Se a conexão também terminar por outro caminho — uma exceção, por exemplo — esses pontos de junção ficam de fora.
- O temporizador nunca para: a falha aparece longe da causa.

### CLASSE 2 — PRECEDÊNCIA INCORRETA




Os dois aspectos afetam o mesmo ponto de junção — o encerramento da conexão. A temporização precisa vir antes, porque o faturamento usa o tempo total. Invertida a precedência, o custo é calculado com o tempo errado.

*Nos dois casos o código compila, cada módulo isolado parece correto e nenhuma classe menciona o aspecto. É esse tipo de defeito que os critérios da Parte 4 procuram tornar visível.*

# 4

# Teste estrutural de programas OA

AODU, critérios transversais, integração contextual e JaBUTi/AJ

# Do bytecode ao grafo AODU

**Bytecode Java**

o modelo de fluxo de dados de Vincenzi et al. parte do bytecode, não do código-fonte

**Grafo de Instrução**

um nó por instrução de bytecode; arestas ligam instruções que podem executar em sequência

**Grafo AODU**

blocos de instruções com informação de definições e usos — o Def-Uso orientado a aspectos

$$AODU(u) = (N, E, s, T, C)$$

- N — nós; s — nó de entrada; T — nós de saída.
- E — arestas, sendo $Ec \subseteq Er$ as arestas transversais, que chegam ou saem de um nó transversal.
- $C \subseteq N$ — nós transversais: blocos em que uma instrução invoca o método correspondente a um adendo.

**UM GRAFO POR UNIDADE**

- método
- método introduzido por declaração intertipos
- adendo

*São essas as três menores partes executáveis de um programa OA — e o teste intramétodo e intra-adendo procura defeitos em cada uma isoladamente.*

# Critérios transversais de unidade

## Todos-Nósc

*Todos-Nós-Transversais*

cada nó transversal — portanto cada execução de adendo na unidade afetada — é exercitado ao menos uma vez.

requisitos no exemplo

`{ 159, 82 }`

## Todas-Arestasc

*Todas-Arestas-Transversais*

cada aresta que tem um nó transversal como origem ou destino é exercitada: um defeito pode se revelar só em um desvio específico.

requisitos no exemplo

`{ (159,230), (82,153), (74,159), (74,82) }`

## Todos-Usosc

*Todos-Usos-Transversais*

para cada variável, um caminho livre de definição da definição até cada uso — inclusive os usos em nós transversais, que são a troca de dados entre classe e aspecto.

requisitos no exemplo

`{ (tc,15,159), (chamado,49,159), (cel,0,82), … }`

*Requisitos derivados do AODU do adendo de contorno do aspecto Transferencia (Tabela 7.1). Para cobrir os dois nós transversais bastam dois casos de teste: um cliente transferido em outra área — longa distância — e outro na mesma área.*

# Integração contextual: o grafo CoDU

Cafeo e Masiero: testar a unidade junto de todas as unidades que se relacionam com ela, até uma profundidade de chamadas e entrecortes escolhida pelo testador.

$$\mathbf{CoDU(u_0,\ d) = (N,\ E,\ S,\ T,\ I,\ Is,\ R)}$$

- Integra ao AODU da unidade em teste os AODU das unidades da cadeia de chamadas e entrecortes.
- Nó integrado, aresta integrada e aresta de integração são os elementos novos; o rótulo do nó traz profundidade.grafo.deslocamento.
- I são os nós de interação (transversais e de chamada); Is, os que serão expandidos.

### CASOS ESPECIAIS NA MONTAGEM

- Métodos de biblioteca não são expandidos.
- Recursão é integrada uma única vez.
- Polimorfismo: o testador escolhe quais métodos integrar (ligação tardia).
- around substitui a unidade; o proceed() reintegra o ponto de junção entrecortado.
- Chamadas múltiplas ao mesmo método são integradas separadamente.

### CRITÉRIOS DE INTEGRAÇÃO (PROFUNDIDADE d)

**Todos-Nós-Integrados-Nd**

cada nó integrado do CoDU, na profundidade d, é exercitado.

**Todas-Arestas-Integradas-Nd**

cada aresta integrada do CoDU, na profundidade d, é exercitada.

**Todos-Usos-Integrados-Nd**

caminhos livres de definição para cada variável de comunicação — parâmetros e retornos — da unidade em teste para as integradas, das integradas para a unidade em teste e entre unidades integradas.

*A profundidade d ($1 \le d \le p$) é o botão de volume do teste de integração: quanto maior, mais contexto — e mais requisitos.*

# JaBUTi/AJ

Extensão da JaBUTi (Java Bytecode Understanding and Testing) para programas AspectJ, implementando os modelos e critérios das seções anteriores.

### Abordagens apoiadas

- teste estrutural de unidade
- teste de integração par a par
- teste de integração contextual

### O que mostra por unidade

- bytecode
- código-fonte, quando disponível
- grafo def-uso com os requisitos destacados

### Como se usa

- gerenciador de projetos escolhe classes e aspectos a instrumentar
- avalia a cobertura de um conjunto de teste existente
- lista requisitos não cobertos para guiar novos casos
- casos de teste em JUnit podem ser importados

*Protótipo operacional acadêmico — como a Proteum/AJ — mas aplicável para elevar a qualidade de sistemas OA também na indústria. A ferramenta ainda resume os requisitos por classe/aspecto, por método/adendo e por critério.*

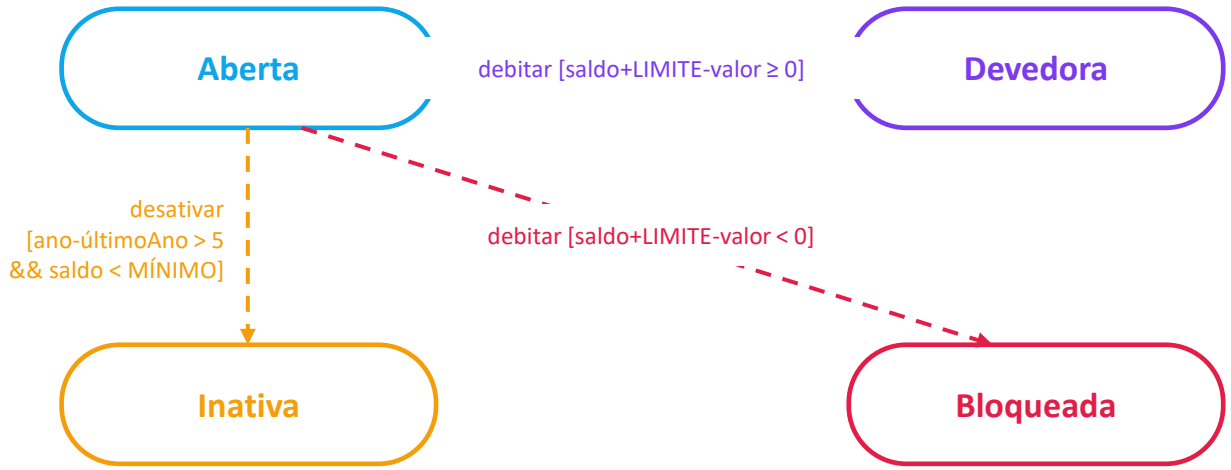
5

# Estados, mutação e ordenação

ASM, operadores de mutação, Proteum/AJ e ordem de integração

# Teste baseado em estados: do FREE ao ASM

O método FREE modela classes com Statecharts da UML; o ASM (Aspectual State Model) estende a notação para representar também os aspectos — apoiando o teste incremental e o teste conjunto de classes e aspectos.




*Tracejado = estado ou transição que só existe por causa dos aspectos. Os estados da classe Conta no modelo FREE original são Aberta, Bloqueada, Inativa e Fechada; as transições, abrir, bloquear, debitar, creditar, obterSaldo, desativar e fechar.*

### O QUE OS ASPECTOS MUDARAM NA CLASSE CONTA

- Saldo negativo dentro de um limite é permitido: surge o estado Devedora, com juros.
- Os débitos passam a ser monitorados — fora do limite, a conta vai para Bloqueada.
- Inativa deixa de depender só dos cinco anos sem movimentação: exige também saldo abaixo do mínimo.

### COMO O ASM MARCA OS PONTOS DE JUNÇÃO

- losango na transição → ponto de junção de entrada
- linha tracejada a partir da transição → ponto de junção de saída
- círculo + X na transição → ponto de junção de contorno

# Árvore de transições e cobertura multicondicional

- A árvore de transições é gerada percorrendo todos os caminhos a partir do estado inicial até uma folha ou de volta ao nó inicial.
- Um nó é terminal quando repete um estado já presente na árvore ou é um estado final.
- Cada caminho da raiz à folha é um requisito de teste; os caminhos tracejados são os que só existem por causa dos aspectos.

Para cobrir Aberta → Devedora → Devedora: abrir(), debitar(valor1), creditar(valor2), com as guardas [saldo+LIMITE-valor ≥ 0] e [saldo+valor < 0] satisfeitas.

Não basta testar o aspecto com as guardas satisfeitas: é preciso exercitar também as condições que não disparam a transição. O critério de cobertura multicondicional exige todas as combinações — e isso acrescenta caminhos à árvore.

**TABELA 7.3 — CONDIÇÕES DA TRANSIÇÃO DESATIVAR**

| Condição | Próximo estado |
|---|---|
| (ano − últimoAno) ≤ 5 && saldo < MÍNIMO | Aberta |
| (ano − últimoAno) ≤ 5 && saldo ≥ MÍNIMO | Aberta |
| (ano − últimoAno) > 5 && saldo < MÍNIMO | Inativa |
| (ano − últimoAno) > 5 && saldo ≥ MÍNIMO | Aberta |

Só uma das quatro combinações leva a Inativa. A árvore original cobre a transição debitar por completo, mas não a desativar: as outras três combinações precisam ser acrescentadas, resultando na árvore de transições expandida.

*Casos de teste são gerados automaticamente da árvore derivada do ASM.*

# Teste de mutação para AspectJ

Modificações no programa geram mutantes; os casos de teste precisam distinguir o original dos mutantes. Os operadores para AspectJ modelam defeitos dos elementos próprios da POA — e os operadores tradicionais de unidade, integração e classes continuam aplicáveis.

**Grupo 1**

### Expressões de conjunto de junção

- PWIW — insere curingas: menos restritivo
- PSWR — remove curingas: mais restritivo
- PWSR / PSSR — troca o tipo por super ou subtipo
- PCTT — this ⇄ target
- PCCE — call / execution / initialization
- PCGS — get ⇄ set
- PCCC — cflow ⇄ cflowbelow
- PCLO — altera os operadores lógicos

**Grupo 2**

### Expressões declare

- DAPC — inverte a ordem em declare precedence
- DAPO — remove declare precedence: ordem arbitrária
- DSSR — remove declare soft
- DEWC — modifica declare error / warning
- DAIC — altera perthis / pertarget / percflow

**Grupo 3**

### Adendos

- ABAR — before ⇄ after [returning|throwing]
- ABHA — remove o adendo por completo
- APSR — remove chamadas a proceed()
- APER — remove as condições que guardam o proceed()
- ABPR — troca o conjunto de junção do adendo
- AJSC — thisJoinPointStaticPart ⇄ enclosing

*Ressalva: defeitos de declarações intertipos são, em geral, detectados na compilação — e o teste de mutação pressupõe mutantes com diferença semântica em tempo de execução. Defeitos do código-base ficam para os operadores tradicionais.*

# Exemplo: o operador PWIW

PWIW insere curingas de AspectJ — “*”, “+” e “..” — no conjunto de junção, tornando-o mais genérico.

**ORIGINAL**

```
pointcut finalizacao(Conexao c): target(c) &&
    call(void Conexao.desconecta());
```

**MUTANTES**

```
call(void *.desconecta());         // mutante 1
call(void Conexao+.desconecta()); // mutante 2
call(* Conexao.desconecta());      // mutante 3
```

Cada mutante potencialmente seleciona mais pontos de junção do que o previsto, ativando o adendo de temporização em momentos indesejados. Cabe ao testador escrever casos de teste que detectem essa ativação e matem os mutantes.

**MUTANTES EQUIVALENTES**

Se o conjunto de junção modificado seleciona exatamente os mesmos pontos de junção do original, o mutante é equivalente — não há caso de teste capaz de matá-lo. A Proteum/AJ detecta automaticamente esses casos nos operadores que atuam sobre conjuntos de junção: no exemplo do livro, 16 mutantes.

**PROTEUM/AJ**

- Família Proteum; modos linha de comando e gráfico.
- Apoia os quatro passos: executar o programa, gerar, executar e analisar os mutantes.
- Compilação e execução descritas em scripts Ant; casos de teste em JUnit.
- Calcula o escore de mutação e permite marcar equivalentes à mão.

# Ordenação de classes e aspectos

Em que ordem testar? A ordem influencia a ordem de implementação, a ordem em que os defeitos aparecem e o número de pseudocontrolados (stubs) e pseudocontroladores (drivers). O problema nasce dos ciclos de dependência.

### ORD — OBJECT RELATION DIAGRAM

- I — herança;  Ag — agregação;  As — associação.
- Em POA não há agregação, mas aparecem dois tipos novos de dependência.
- ORD estendido: aresta P (conjuntos de junção) e aresta IT (declarações intertipos).

### ESTRATÉGIA DE BRIAND ET AL.

- Algoritmo de Tarjan sobre o ORD para achar componentes fortemente conexos (CFC).
- Nos CFCs não triviais, remover a aresta de associação de maior peso para quebrar o ciclo.
- Peso = arestas de entrada da origem × arestas de saída do destino.
- Repetir Tarjan até só restarem CFCs triviais.

```
H→B = 3×3 = 9      A→C = 3×3 = 9      C→E = 3×2 = 6
C→H = 3×2 = 6      E→F = 2×2 = 4      B→C = 1×3 = 3
C→A = 3×1 = 3      B→D = 1×2 = 2      E→A = 2×1 = 2
```

*No exemplo do livro, remover A→C ou H→B é indiferente: mesmo peso, mesmo número de pseudocontrolados.*

### APLICADA AO ORD ESTENDIDO

- Tendência geral: desenvolver e testar as classes antes dos aspectos.
- Com aspectos todos transversais, a estratégia incremental se aplica sem ajustes e minimiza os pseudocontrolados.
- Ciclos entre aspectos também existem — e aí é preciso implementar aspectos pseudocontrolados.

# Quando a classe conhece o aspecto

**Inconsciência incompleta.** Há casos em que as classes têm conhecimento dos aspectos que as entrecortam — persistência e controle de transações são os exemplos clássicos. Isso gera dependência circular, aqui tratada como dependência bidirecional, e a receita “classes primeiro” deixa de servir.

## 1 Classes primeiro, com implementação alternativa

- Uma ou mais classes ganham versão provisória — objetos só em memória, sem persistência.
- Na integração do aspecto, desfaz-se o que foi feito e tudo é testado de novo.
- Pode custar mais que criar aspectos pseudocontrolados: há retrabalho e teste de regressão.

## 2 Aspectos primeiro

- Briand mostra que os aspectos bidirecionais podem ser desenvolvidos e testados logo no início, antes de qualquer classe consciente deles.
- Gera mais pseudocontrolados, mas evita retrabalho e regressão.
- Eficiente quando há framework pronto para o interesse — basta instanciá-lo no momento certo.

## 3 Quebrar a dependência

- Transformar o aspecto em dois, cada um dependente da classe.
- A classe volta a não ter consciência dos aspectos e pode ser desenvolvida e testada primeiro.
- Se classes e aspectos forem desenvolvidos juntos, o algoritmo de Briand ordena os dois, minimizando os pseudocontrolados.

# Considerações finais

## 1 POA a favor do teste

- A POA separa o código-base do código de infraestrutura de teste.
- Imitadores virtuais, teste embutido, verificação de padrões, contratos e instrumentação.
- Apoio ao teste em vários estágios, inclusive o teste de regressão — tópico ainda em pesquisa.

## 2 Teste do que usa POA

- Técnicas de programas procedimentais e OO adaptadas: estrutural, baseada em estados e baseada em defeitos.
- Modelos e critérios próprios (AODU, CoDU) e operadores de mutação para construções da POA.
- Ferramentas em protótipo acadêmico: JaBUTi/AJ e Proteum/AJ.

**Tendências apontadas pelos autores:** teste baseado em conjuntos de junção, teste de integração contextual e teste de mutação — convencional e com custo reduzido.

*A conclusão do capítulo é simétrica: a mesma propriedade que faz do aspecto um bom lugar para o código de teste — alcançar muitos pontos sem ser chamado — é o que exige modelos de teste próprios quando o aspecto é o código testado.*

# Obrigado!

## Perguntas?

LEMOS, O. A. L.; RÉ, R.; MASIERO, P. C.; FERRARI, F. C.; CAFEO, B. B. P. Teste de aspectos e teste apoiado por aspectos.
In: DELAMARO, M. E.; MALDONADO, J. C.; JINO, M. (org.). Introdução ao teste de software. 2. ed. Rio de Janeiro: Elsevier, 2016. cap. 7.

Bernardo Vargens Broedel · Eduardo Capobiango Ferraz · Mateus Biancardi da Silva · Miguel Zon Murad · Victor Augusto Fernandes Rangel